

Guía Docente — Sesión 3: Docker Compose y Aplicaciones Multi-Contenedor

Curso: Docker desde Cero: Crea y Despliega Aplicaciones (10ma Edición 2026) **Docente:** Ing. Cristian Jampier Chileno Segundo | OTI - UNI **Programa:** Programa de Iniciación Tecnológica (PIT) 2026

Perfil del Alumno y Enfoque Pedagógico

Los estudiantes entienden el ciclo de vida pero les resulta complejo coordinar múltiples contenedores manuales (Flask + PostgreSQL). Presenta Docker Compose como un orquestador local declarativo en YAML. Explica las reglas de indentación de YAML, el concepto de Service Discovery (DNS interno de Docker) y la segregación segura de secretos mediante archivos .env.

Planificación de la Clase (3 Horas)

Bloque	Tiempo	Contenido
Introducción	00:00 - 00:30	El problema de apps multi-contenedor y transición de run a compose
Estructura YAML	00:30 - 01:10	Piezas de compose.yaml: services, networks, volumes y depends_on
Comandos Compose	01:10 - 01:40	up -d, ps, logs -f, exec y down
Receso	01:40 - 01:55	Break
Flask + Postgres	01:55 - 02:40	Resolución DNS interna, configuración de secretos con .env
Cierre	02:40 - 03:00	Laboratorio Práctico, Tarea y Q&A

Guión Paso a Paso del Docente

Introducción: Cómo comenzar la clase

Guión Sugerido (Lo que debes decir): "Buenas tardes. Hasta ahora hemos levantado contenedores individuales. Pero en producción, una aplicación no vive sola: requiere base de datos, proxies y caché. Hoy aprenderemos a unificar todo nuestro stack usando Docker Compose, declarándolo en un único archivo YAML y controlándolo con comandos simples."

Punto 1: El problema de multi-servicios y docker run repetitivo

Guión Sugerido (Lo que debes decir): "Para levantar Flask conectado a Postgres de forma manual tendríamos que crear una red, levantar Postgres con variables de entorno, y luego levantar Flask enlazado a la misma red. Correr esto de forma imperativa es lento y propenso a fallas.

Docker Compose viene a resolver esto guardando la topología completa en un archivo descriptivo llamado `docker-compose.yml` (o `compose.yml`)."

Diagrama de la PPT:

```
compose.yml → docker compose → web + db + red + volumen
```

Punto 2: Estructura del YAML de Compose y comandos esenciales

Guión Sugerido (Lo que debes decir): "El archivo YAML de Compose se compone de 4 bloques lógicos principales: `services` (los contenedores a correr), `networks` (comunicación interna aislada), `volumes` (almacenamiento persistente) y `depends_on` (orden de encendido).

OJO: `depends_on` solo define qué contenedor prende primero; NO garantiza que la BD esté lista para aceptar conexiones. Para eso usaremos `healthchecks` en la sesión 4."

Estructura YAML mínima para proyectar:

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass
```

Comandos esenciales de Compose:

Comando	Descripción
<code>docker compose up -d</code>	Levantar en segundo plano
<code>docker compose up -d --build</code>	Levantar reconstruyendo imágenes
<code>docker compose ps</code>	Ver servicios activos
<code>docker compose logs -f</code>	Ver logs en vivo

Comando	Descripción
<code>docker compose down</code>	Detener y eliminar contenedores
<code>docker compose down -v</code>	Detener y eliminar + volúmenes
<code>docker compose exec web sh</code>	Entrar al contenedor web

Punto 3: Service Discovery y DNS interno de Docker

Guión Sugerido (Lo que debes decir): "Cuando levantamos el stack, Docker Compose crea una red interna y habilita un resolvidor de DNS en la IP loopback `127.0.0.11` dentro de cada contenedor.

Esto es CLAVE: en la configuración de Flask, en vez de usar 'localhost' o IPs variables, nos conectamos a la BD usando el nombre lógico del servicio: 'db'. El resolvidor de Docker intercepta la búsqueda DNS y responde con la IP virtual del contenedor Postgres.

Además, por seguridad (Hardening), eliminamos el mapeo de puertos ('ports') de la base de datos para no exponerla a internet pública."

Ejemplo de conexión en Python:

```
# CORRECTO: usar el nombre del servicio como hostname
conn = psycopg2.connect(host="db", database="appdb", user="appuser",
password="apppass")

# INCORRECTO: no usar localhost ni IPs fijas
conn = psycopg2.connect(host="localhost", ...) # ✗
```

Punto 4: Segregación de secretos con .env y .env.example

Guión Sugerido (Lo que debes decir): "No debemos dejar contraseñas quemadas en compose.yaml. Creamos un archivo local `.env` (excluido de Git con `.gitignore`) que almacene las variables. En el YAML hacemos referencia usando `${VAR}`."

Subiremos a Git únicamente una plantilla llamada `.env.example` con las variables vacías para guiar a otros desarrolladores. Esto protege las credenciales de producción de ser subidas a repositorios públicos."

Ejemplo:

```
# .env (LOCAL, NUNCA en Git)
POSTGRES_DB=appdb
POSTGRES_USER=appuser
POSTGRES_PASSWORD=SuperSecreta123!

# .env.example (SÍ en Git)
POSTGRES_DB=
```

```
POSTGRES_USER=  
POSTGRES_PASSWORD=
```

💬 Dinámica e Interacción en el Aula

- **Pregunta para lanzar al aula:** "¿Qué pasa con los datos de tu base de datos si ejecutas 'docker compose down'? ¿Se pierden para siempre?"
 - **Respuesta:** Depende. `docker compose down` borra contenedores y redes, pero NO borra volúmenes nombrados. `docker compose down -v` Sí borra los volúmenes y los datos se pierden. Lo veremos en la sesión 4.
- **Pregunta técnica:** "¿Flask puede conectarse a Postgres usando 'localhost'?"
 - **Respuesta:** No. `localhost` dentro del contenedor Flask apunta a sí mismo, no a Postgres. Debe usar el nombre del servicio `db` como hostname.

? FAQs

1. **¿Por qué YAML y no JSON para Compose?**
 - YAML es más legible para humanos y soporta comentarios. Docker Compose usa YAML por convención.
2. **¿Puedo tener dos archivos compose en el mismo directorio?**
 - Sí, usando el flag `-f`: `docker compose -f compose.yml -f compose.override.yml up -d`.
3. **¿Qué pasa si no creo el archivo .env?**
 - Las variables `${VAR}` quedarán vacías y los servicios pueden fallar al iniciar.

🔑 Práctica en Consola Paso a Paso

Lab 1: Preparar variables

```
cd codigo/sesion3  
cp .env.example .env  
# En Windows PowerShell:  
# Copy-Item .env.example .env
```

Lab 2: Levantar el stack

```
docker compose up -d --build  
docker compose ps  
docker compose logs -f
```

Probar en `http://localhost:5000`

Lab 3: Explorar el stack

```
docker compose exec web sh
docker compose exec db psql -U appuser -d appdb
# Dentro de psql:
SELECT version();
\q
```

Lab 4: Modificar y reconstruir

```
# Editar app.py, cambiar el mensaje
docker compose up -d --build
# Verificar el cambio en http://localhost:5000
```

Lab 5: Apagar y limpiar

```
docker compose down
docker compose down -v # también borra volúmenes
```

Trabajo para el Hogar

1. Levantar el stack Flask + PostgreSQL con Docker Compose.
2. Verificar que la app Flask se conecta correctamente a PostgreSQL.
3. Modificar el archivo `app.py` para que muestre un mensaje personalizado.
4. **Entregable:** Captura de pantalla de `docker compose ps` mostrando ambos servicios activos y del navegador con la respuesta de la app.

Gestión del Aula y Errores Comunes

Error	Solución
<code>ERROR: yaml: line X: did not find expected key</code>	Error de indentación en YAML. Usar ESPACIOS, nunca tabuladores
<code>db connection refused</code>	La BD aún no está lista. <code>depends_on</code> no espera readiness. Agregar healthcheck (sesión 4)
<code>.env not found</code>	Crear el archivo <code>.env</code> copiando <code>.env.example</code>
<code>port already in use</code>	Otro servicio usa el puerto 5000. Cambiar a <code>-p 5001:5000</code>

Evaluación — Test de la Sesión 3 (12 Preguntas)

1. ¿Qué es Docker Compose en el ecosistema de contenedores?

- a) Un comando nativo del Kernel de Linux para crear redes VLAN.
- b) Una herramienta para definir y ejecutar aplicaciones Docker de múltiples contenedores de forma declarativa mediante archivos YAML.
- c) Un compilador de código fuente para lenguajes web.
- d) Un registro en la nube que almacena contenedores inactivos.

2. ¿Cuál es la regla de sintaxis más importante en los archivos YAML de Docker Compose?

- a) Separar todas las directivas usando puntos y comas.
- b) Usar obligatoriamente llaves `{ }` para encerrar cada servicio.
- c) Respetar estrictamente la indentación usando espacios en blanco (nunca tabuladores).
- d) Escribir todos los comandos en una sola línea continua.

3. ¿Cuál sección de un archivo `compose.yaml` declara los contenedores que se ejecutarán?

- a) `volumes`
- b) `services`
- c) `networks`
- d) `configs`

4. Si tienes un servicio `db` y un servicio `web`, ¿cómo se comunica Flask internamente con la BD?

- a) Usando la dirección IP dinámica de la máquina Host.
- b) Resolviendo el hostname lógico `db` provisto por el DNS interno de Docker en la red común.
- c) Abriendo un canal SSH interactivo.
- d) Mapeando los puertos locales a través de sockets UNIX compartidos.

5. ¿Qué comando de Docker Compose inicia todos los servicios en segundo plano?

- a) `docker compose start`
- b) `docker compose up -d`
- c) `docker compose run`
- d) `docker compose deploy --all`

6. ¿Qué directiva del archivo Compose establece el orden de inicio de los servicios?

- a) `requires`
- b) `depends_on`
- c) `links_to`
- d) `after`

7. ¿Dónde se deben almacenar los secretos y contraseñas de bases de datos?

- a) Directamente quemados (hardcoded) en `compose.yaml`.
- b) En un archivo local `.env` excluido del control de versiones.
- c) En la carpeta `/tmp` del host.
- d) En la documentación pública del README.md.

8. ¿Qué comando de Compose detiene todos los contenedores y elimina las redes del proyecto?

- a) `docker compose stop`

- b) `docker compose down`
- c) `docker compose rm`
- d) `docker compose clean`

9. ¿Qué utilidad tiene subir un archivo `.env.example` al repositorio Git?

- a) Funcionar como plantilla del archivo `.env` mostrando qué variables configurar, sin revelar secretos.
- b) Cifrar los logs de salida del contenedor en producción.
- c) Descargar las imágenes oficiales del registro.
- d) Ejecutar pruebas unitarias automatizadas.

10. Si deseas ver los logs de todos los servicios en tiempo real, ¿qué comando corres?

- a) `docker compose logs -f`
- b) `docker compose inspect logs`
- c) `docker compose ps --logs`
- d) `docker compose stats`

11. ¿Qué diferencia hay entre `docker compose down` y `docker compose down -v`?

- a) No hay diferencia, ambos hacen lo mismo.
- b) `docker compose down -v` también elimina los volúmenes nombrados, borrando datos persistentes como bases de datos.
- c) `docker compose down -v` solo detiene los contenedores sin eliminarlos.
- d) `docker compose down -v` activa el modo verbose para ver más detalles.

12. ¿Por qué Flask dentro de un contenedor NO puede conectarse a PostgreSQL usando `localhost`?

- a) Porque PostgreSQL no soporta conexiones locales.
- b) Porque `localhost` dentro del contenedor Flask apunta a sí mismo, no al contenedor de PostgreSQL. Debe usar el nombre del servicio (`db`).
- c) Porque Docker bloquea todas las conexiones localhost por seguridad.
- d) Porque localhost solo funciona con bases de datos MySQL.

 Solucionario Sesión 3

1-b | 2-c | 3-b | 4-b | 5-b | 6-b | 7-b | 8-b | 9-a | 10-a | 11-b | 12-b